



High Level Design (HLD)

Testing Professor · Playwright BDD Cucumber Framework

Version 1.4 · TypeScript · Updated 16 Aug 2026 · Stakeholder briefing pack also available as Word and PowerPoint

0. Executive summary (for stakeholders)

What it is: A reusable UI test automation platform that describes business flows in plain English (Gherkin), drives a real browser with Playwright, and produces branded HTML reports, videos, and optional AI failure insights.

What it tests today: The core banking application at <https://corebanking.testingprofessor.net/> — staff login, bank-manager customer lifecycle (add / search / open accounts), and customer deposit journeys.

Why it matters: Faster regression feedback, living documentation of business rules, consistent evidence (screenshots, video, traces), and CI that can run against the real AUT on Chromium, Firefox, and WebKit.

Delivery: Local runs, Docker, Jenkins, and GitHub Actions against the real AUT. Credentials are never committed — they come from `.env` locally or CI secrets in the pipeline.

1. Purpose

This framework is a reusable UI automation platform using Playwright and Cucumber BDD. It standardizes Gherkin features, thin step definitions, business classes, page objects, a rich action utility (interactions, frames, shadow DOM, uploads/downloads, scrolling, JavaScript), optional self-healing locators, wait/assertion helpers, logging, error classification, env-driven artifacts (step screenshots, video, traces), Testing Professor–branded Extent-style activity reporting, CI/CD, BrowserStack, database-backed history, and local LLM failure triage with complexity-based model routing.

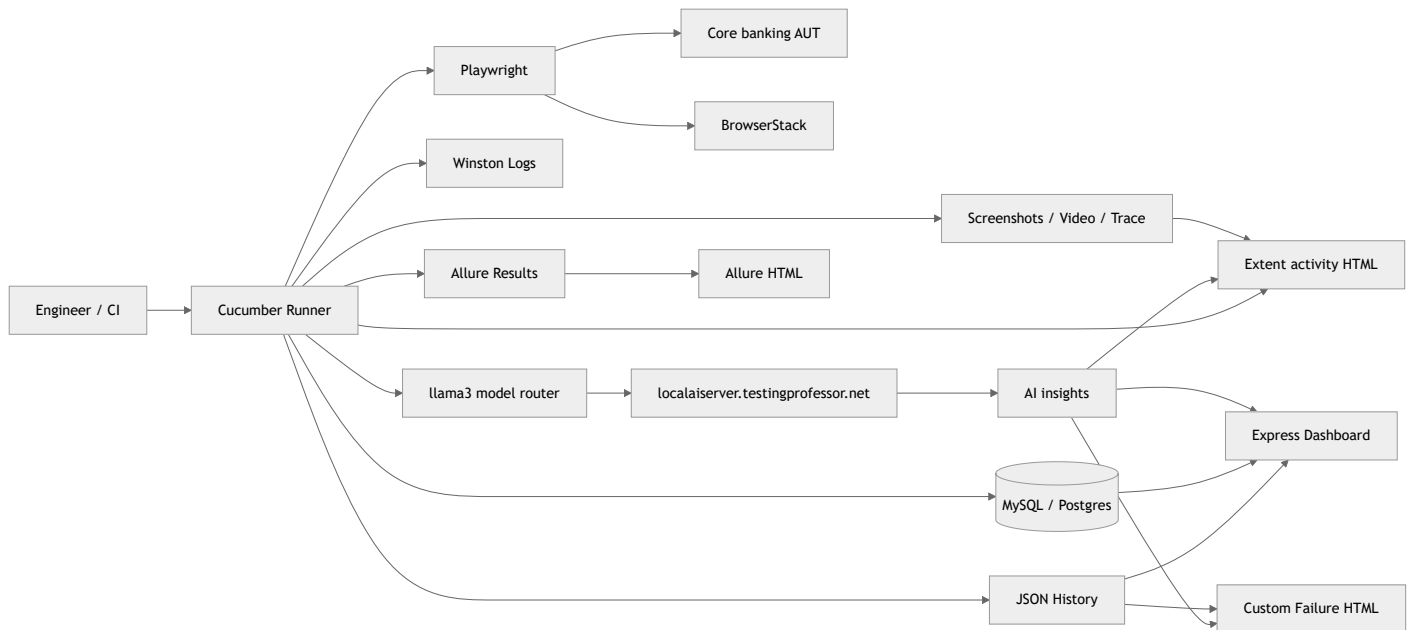
The current application under test is the core banking site at <https://corebanking.testingprofessor.net/>.

2. Business outcomes and goals

Outcome	How the framework supports it
Readable tests for BA / QA / Dev	Gherkin features; thin steps call business classes
Maintainable automation	Page Object Model + single PlaywrightActions utility
Faster failure analysis	Extent activity log, custom failure HTML, optional LLM triage
Evidence for audits / demos	Screenshots, scenario video, Playwright traces, branded reports
CI confidence	GitHub Actions / Jenkins / Docker against real AUT
Resilience	Optional self-heal; DB and LLM are optional add-ons

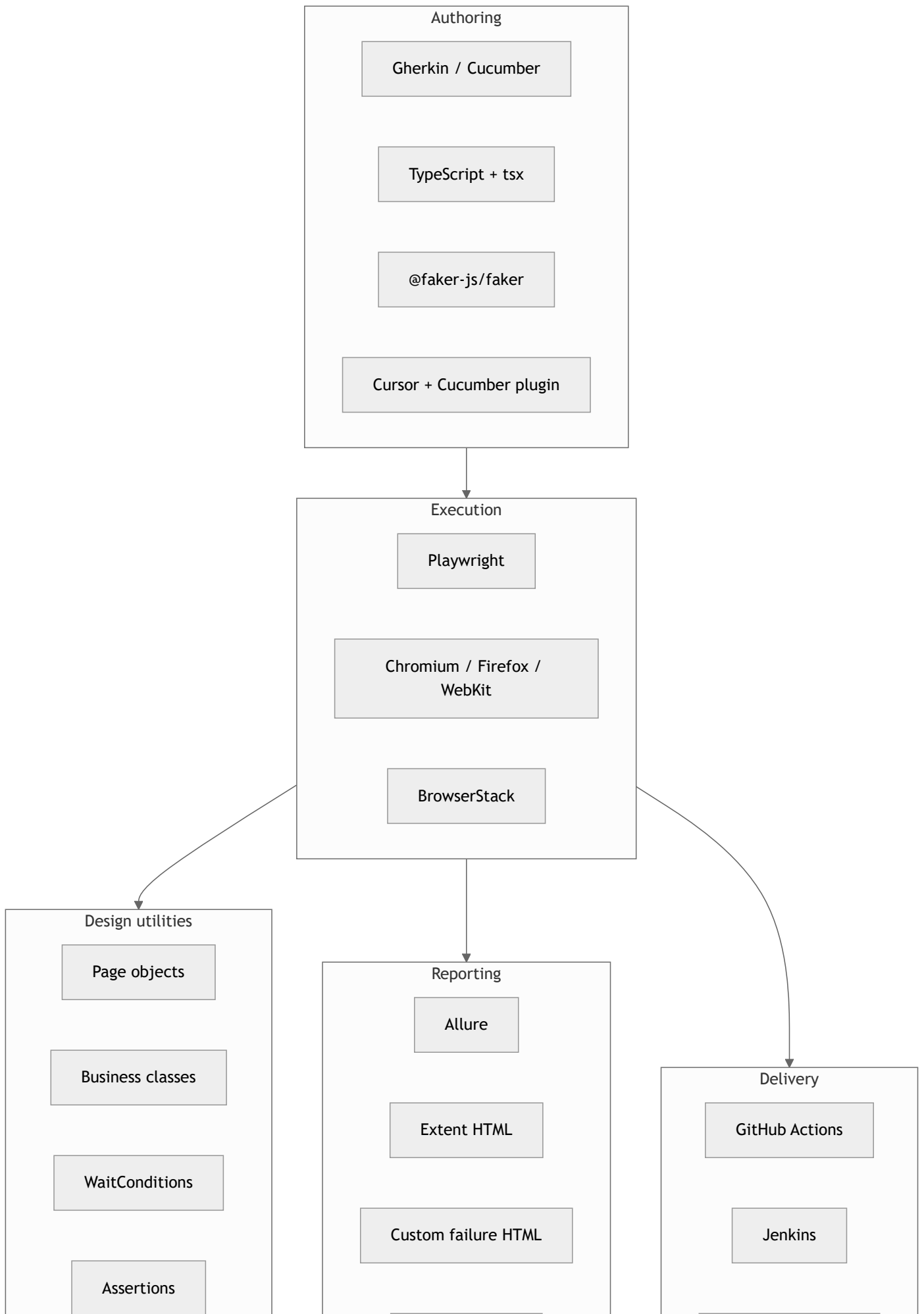
- BDD-first authoring with Gherkin features and thin step definitions that call business classes.
- Page Object Model with a single action utility wrapping Playwright APIs (including drag/drop, frames, shadow, file I/O, scroll, and `evaluate`).
- Optional self-heal for broken locators (`SELF_HEAL_ENABLED`): cache → heuristics → optional LLM.
- Deterministic error taxonomy wrapping Playwright exceptions.
- Worker-safe shared data for parallel and cross-browser runs.
- Allure, custom failure HTML, Extent activity report (Testing Professor branded), and historical dashboard.
- Env-driven per-step screenshots, scenario screenshots, video under `recordings/test-runs/`, and Playwright traces.
- Optional MySQL or PostgreSQL persistence.
- GitHub Actions, Jenkins, Docker, and BrowserStack integration against the real AUT.
- Hosted Ollama LLM triage: llama3 routes work; complex analysis prefers ~32B models, then `gemma4:latest`.

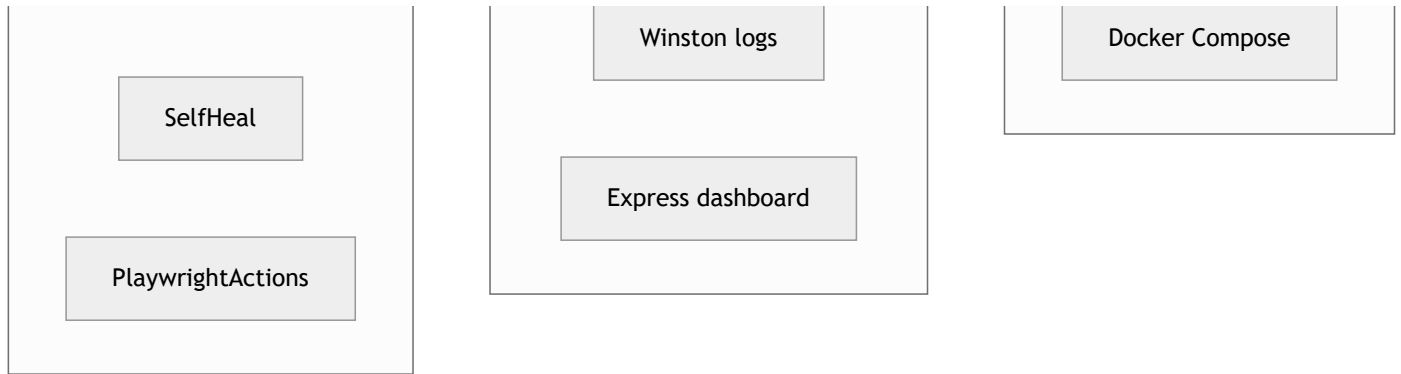
3. System context



4. Automation testing tools

The stack below is what this project uses to author, execute, observe, and report UI automation. Cucumber is the test runner; Playwright is the browser library (Playwright Test is not the runner).





Category	Tool	Role in this project
Language and runtime	Node.js 24 LTS+	Runtime for Cucumber, Playwright, dashboard, and scripts
	TypeScript 7	Typed features, steps, business classes, and page objects
	tsx	Executes TypeScript directly (hooks, reports, dashboard, docs)
	dotenv / cross-env	Environment config and cross-platform test scripts
BDD and automation	Cucumber.js (@cucumber/cucumber)	Test runner: Gherkin features, tags, parallel, formatters
	Gherkin	Business-readable scenarios in <code>features/*.feature</code>
	Playwright 1.62.1	Browser automation, codegen, traces, video, headed/headless
	Playwright browsers	Chromium/Chrome, Firefox, and WebKit coverage
Design and data	Page Object Model	Locators and UI actions per screen
	Business classes	Flow orchestration called from thin step definitions
	PlaywrightActions	Clicks, drag/drop, frames, shadow DOM, upload/download, scroll, JS evaluate

	WaitConditions / Assertions	Shared waits, observe-pause after button clicks, assertions
	Self-heal (SELF_HEAL_ENABLED)	Optional runtime locator recovery: cache → heuristics → optional LLM
	@faker-js/faker	Random customer and deposit test data
Reporting and logs	Allure (allure-cucumberjs + CLI)	Standard test report from Cucumber results
	Extent-style HTML	Branded step/action activity, screenshots, video, traces, AI notes
	Custom failures.html	Failure-focused HTML for root-cause review
	Winston	Structured console and file logging
	Express dashboard	Historical runs, trends, and AI insights on port 3000
Environments	BrowserStack	Cloud browsers when EXECUTION_ENV=browserstack
	Docker Compose	MySQL, Postgres, optional Ollama, dashboard, test profile
	GitHub Actions / Jenkins	CI/CD pipelines for repeatable runs
Supporting	MySQL / PostgreSQL	Optional persistence of runs, failures, and insights
	Ollama / hosted LLM	Failure triage and model routing
	Cursor Cucumber extension	Gherkin highlighting, step autocomplete, go-to-definition

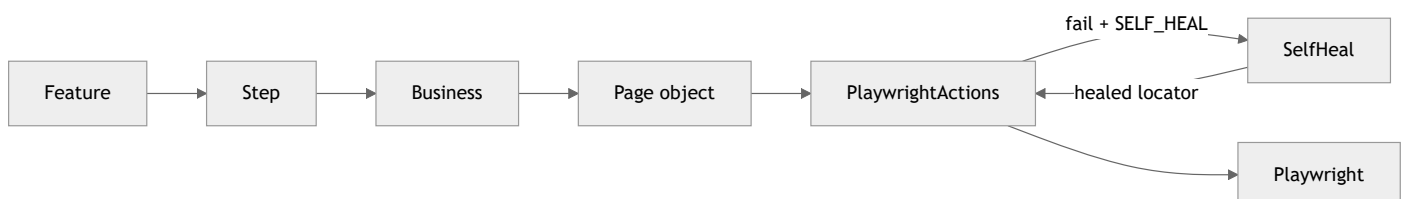
5. Logical architecture

Layer	Components	Responsibility
Authoring	features/*.feature, features/steps	Business-readable scenarios; steps only call business methods

Business	LoginBusiness, CustomerBusiness, ApplicationBusiness	Orchestrate flows, shared buffer, page-object calls
Runtime	CustomWorld, hooks, Playwright browser/context/page	Lifecycle, artifacts, correlation IDs; exposes <code>this.login</code> / <code>this.customer</code> / <code>this.application</code>
POM	BasePage, LoginPage, AddCustomerPage, CustomersPage, OpenAccountPage, CustomerDashboardPage	Locators and page behaviors for core banking
Actions	PlaywrightActions, WaitConditions, Assertions, SelfHeal	UI verbs, waits, asserts, optional locator healing, mapped errors
Artifacts	hooks + SCREENSHOT / STEP_SCREENSHOT / VIDEO / TRACE	Capture, retain-on-failure, attach to reports; downloads under <code>reports/downloads/</code>
Cross-cutting	Logger, SharedBuffer, FrameworkError, optional manager session	Observability, data sharing, taxonomy, auth helpers
Persistence	DB adapters, history JSON	Run/scenario/failure/insight storage
Insights	IlmClient, modelRouter, FailureTriage	Catalog models, route by complexity, classify failures
Presentation	Allure, custom HTML, Extent HTML, dashboard	Step activity, screenshots, video, traces, AI notes
Delivery	GitHub Actions, Jenkins, Docker	Repeatable CI/CD against core banking AUT

5.1 Call chain

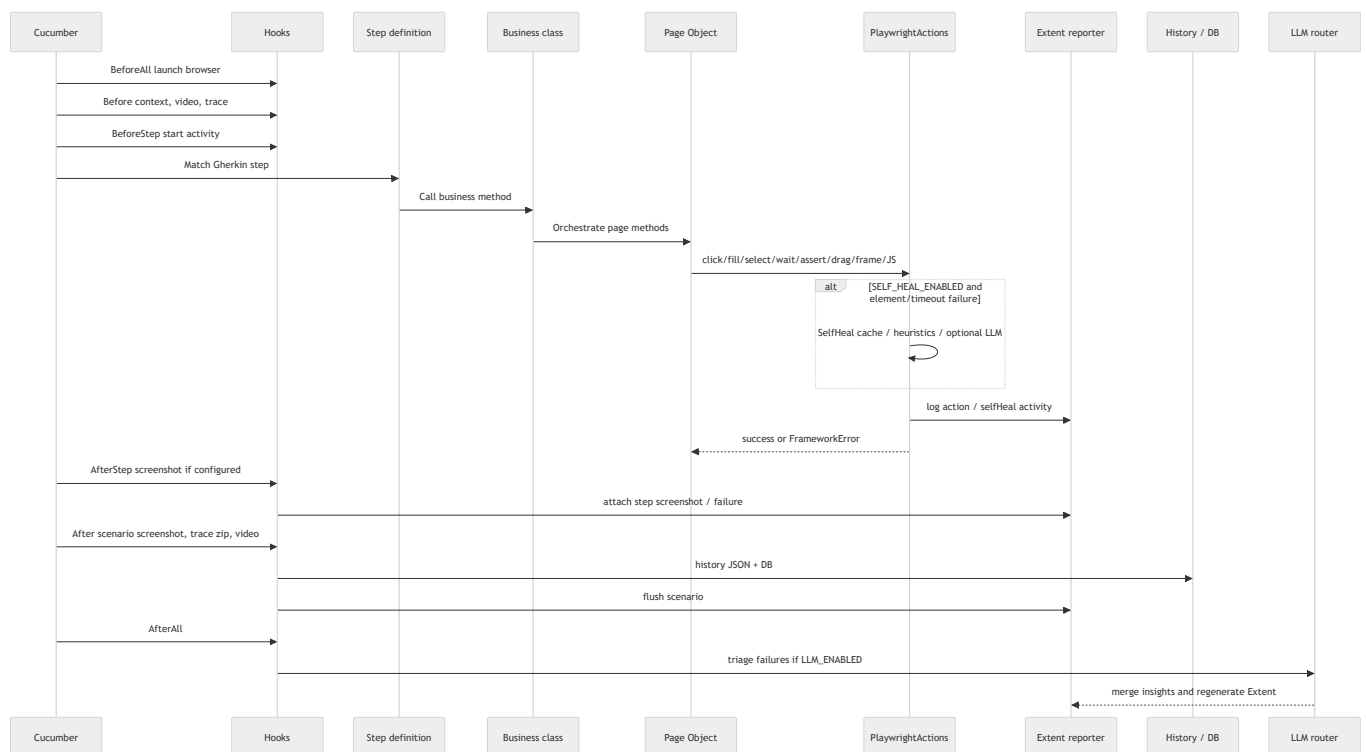
Feature → Step → Business → Page → PlaywrightActions → Playwright. Self-heal sits inside PlaywrightActions when enabled.



5.2 Current automation scope

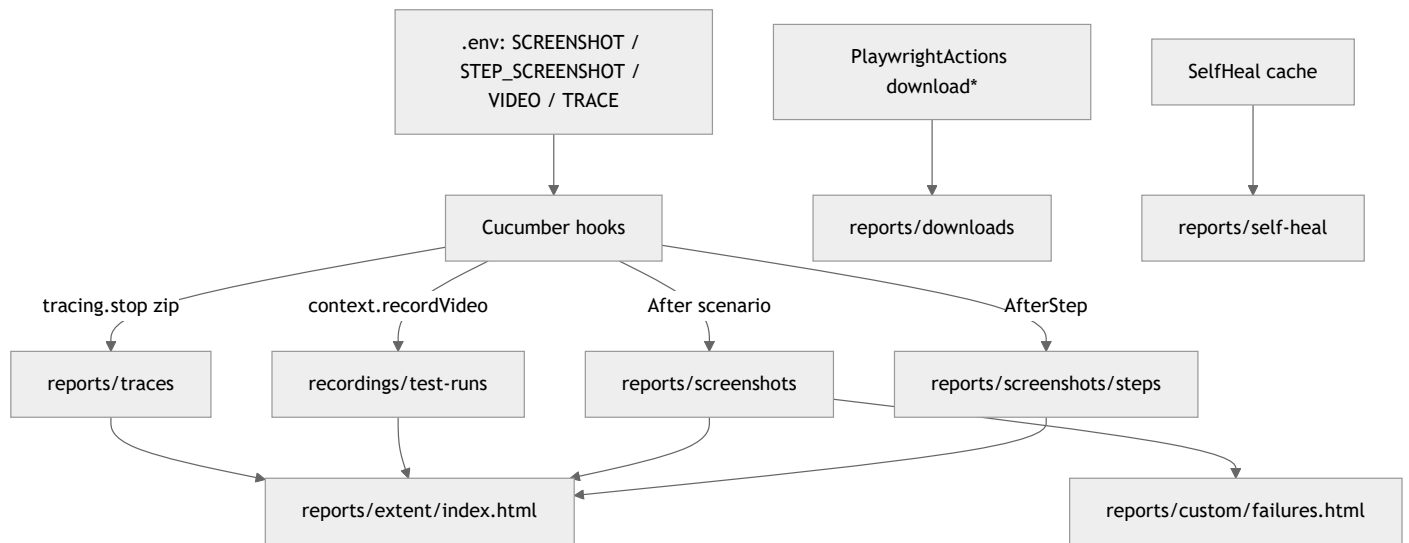
Area	Coverage
Login	Staff sign-in with encrypted credentials from <code>.env</code> / CI secrets
Customers	Add customer, search (first name), open accounts, data-driven examples
Customer portal	Continue as customer, deposit flows (e2e tags)
Quality tags	@smoke, @positive, @negative, @datadriven, @e2e

6. Execution flow



7. Artifact capture

All artifact modes are `off`, `on`, or `retain-on-failure`, driven from `.env`.



Flag	Scope	Output
<code>STEP_SCREENSHOT</code>	After every Gherkin step	<code>reports/screenshots/steps/</code> , inline in Extent
<code>SCREENSHOT</code>	End of scenario	<code>reports/screenshots/</code>
<code>VIDEO</code>	Browser context recording	<code>recordings/test-runs/*.webm</code> , player + link in Extent
<code>TRACE</code>	Playwright trace	<code>reports/traces/*.zip</code>
<code>SELF_HEAL_ENABLED</code>	Action retries	<code>reports/self-heal/healed-locators.json</code> + Extent <code>selfHeal</code> activities

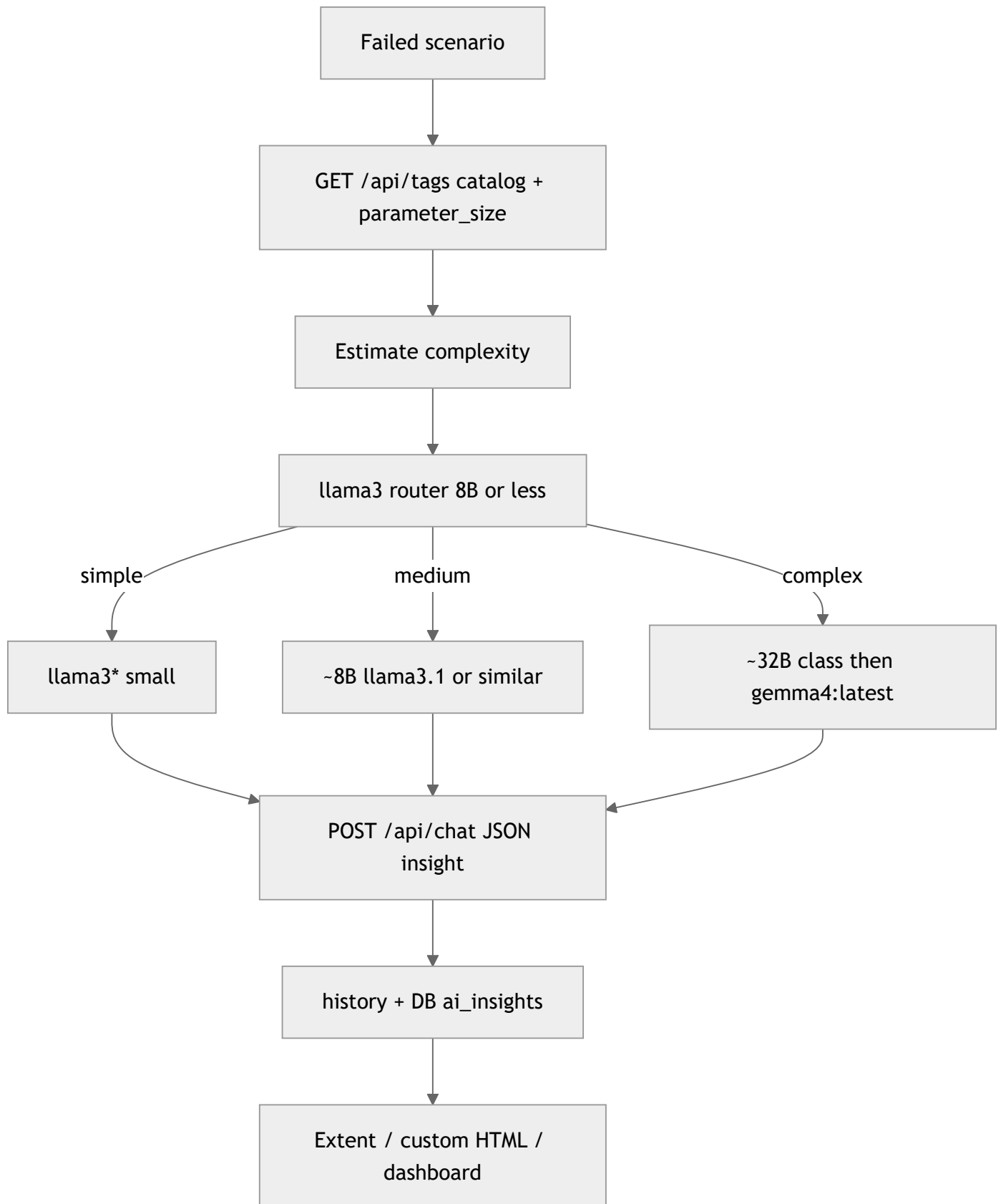
7.1 Action capability groups

Group	Examples in PlaywrightActions
Core UI	click, dblClick, fill, type, clear, select, hover, check/uncheck
Drag and drop	dragAndDrop, dragAndDropByMouse, dragByOffset
Frames	frame, nestedFrame, inFrame, clickInFrame, fillInFrame, getFrameByUrl/Name
Shadow DOM	inShadow, shadowLocator, deepShadowLocator, clickInShadow, fillInShadow

Files	upload, uploadPayload, clearUpload, download*, downloadAndSave
Scroll	scrollIntoView, scrollBy/To, scrollToTop/Bottom, scrollElement*, mouseWheel
JavaScript	evaluate, evaluateScript, evaluateOnLocator, evaluateHandle, dispatchDomEvent, setValueByJs
Windows	launchApplicationInNewWindow, clickAndSwitchToNewWindow*, switchToWindow*

8. LLM triage and model routing

The hosted Ollama server is `http://localaiserver.testingprofessor.net`. Calls use `LLM_API_KEY`.
Catalog comes from `GET /api/tags` (including `details.parameter_size`).



Complexity	Preferred model
Simple (short timeout / locator miss)	Small llama3* (8B or less), never a 32B model

Medium	~8B class unless the failure looks ambiguous
Complex (long stack, PW_UNKNOWN)	~32B installed model (27B–40B band), else <code>gemma4:latest</code> , else larger

`LLM_MODEL=auto` and `LLM_ROUTE=true` enable routing. `LLM_PREFER_BILLION=32` sets the complex-size target. `LLM_COMPLEX_MODEL=gemma4:latest` is the named fallback.

9. Deployment views

9.1 Local

Node 24 LTS+, Playwright browsers, optional MySQL/Postgres. Dashboard on port 3000. LLM defaults to the hosted server `localaiserver.testingprofessor.net` (not a local Ollama process).

9.2 Docker Compose

Services: `mysql`, `postgres`, `ollama` (optional local), `dashboard`, `tests` (profile). Tests write reports to a bind-mounted volume. Hosted LLM can replace compose Ollama via `LLM_HOST`.

9.3 Cloud and CI

When `EXECUTION_ENV=browserstack`, the runner connects via Playwright CDP WebSocket using BrowserStack capabilities instead of launching a local browser.

GitHub Actions runs a browser matrix (Chromium, Firefox, WebKit) against `https://corebanking.testingprofessor.net/` with `LLM_ENABLED=false` and tags such as `@smoke` and `not @example`.

9.4 CI secrets (required for authenticated runs)

The workflow verifies credentials before tests and fails early if they are missing. Configure these as **repository secrets** (Settings → Secrets and variables → Actions) — do not commit them.

Secret	Required	Purpose
<code>APP_USERNAME</code>	Yes	Core banking login email / username
<code>APP_PASSWORD</code>	Yes	Plain password, or <code>enc.v1...</code> ciphertext when encryption is used

APP_ENCRYPTION_KEY	Only if password is encrypted	Decrypts APP_PASSWORD at runtime
BROWSERSTACK_USERNAME / BROWSERSTACK_ACCESS_KEY	Optional	Only for the BrowserStack workflow job
LLM_API_KEY	Optional	Not used in default CI (LLM_ENABLED=false)

Optional repository **variable**: BASE_URL (defaults to the core banking AUT above).

10. Data sharing

Cucumber workers do not share Node memory. SharedBuffer exposes three scopes:

- **scenario** — in-memory Map on World (isolated).
- **worker** — in-memory Map per worker process.
- **global** — locked JSON file plus optional shared_kv table so Chrome/Firefox/WebKit and CI shards can exchange keys.

Customer flows persist customer, customers, accounts in scenario and global scopes so later steps (open account, customer login) can reuse generated data.

11. Reporting strategy

Channel	When	Audience
Winston files/console	Every action/step	Debugging
Allure	Each run	QA / CI artifacts
Extent activity HTML	End of run (and after triage)	Step + Playwright action log, screenshots, video, trace, AI note (Testing Professor branded)
Custom failures.html	Post-run / after triage	Root-cause review (branded)
Dashboard	Historical	Leads / trends / flaky tests / AI insights (branded)

AI insights	AfterAll when enabled, or dashboard / <code>npm run ai:triage</code>	Triage acceleration
-------------	--	---------------------

12. Non-functional requirements

- Isolation: new browser context per scenario.
- Parallel: Cucumber `--parallel` with file-locked global store; AfterAll triage runs on the primary worker only.
- Resilience: DB, LLM, and self-heal are optional; JSON history and Extent still generate.
- Security: credentials and `LLM_API_KEY` only via environment / CI secrets, never committed.
- Extensibility: add a page class + business method + feature + thin step without changing core.
- Observability: button clicks may pause (`CLICK_PAUSE_MS`) for headed runs; CI sets pauses to 0.

13. Constraints and assumptions

- TypeScript 7 on Node 24 LTS or later, executed with `tsx` (not `ts-node`).
- Playwright 1.62.1 library (Cucumber is the test runner; Playwright Test is not used as the runner).
- Core banking pages are implemented; `templates/ExamplePage` remains a scaffold for new pages.
- Open shadow DOM is pierceable; closed shadow roots are not.
- Self-heal is off by default; LLM heal requires `SELF_HEAL_USE_LLM` plus `LLM_ENABLED` and an API key.
- The local AI server requires a valid `LLM_API_KEY`. HTTPS to that host is not used; HTTP is.
- 32B-class models are detected from Ollama catalog metadata or names such as `:32b`.

14. Stakeholder deliverables

Format	File	Use
HTML	<code>docs/hld.html</code>	Interactive design with diagrams
PDF	<code>docs/hld.pdf</code>	Print / archive
Word	<code>docs/hld-stakeholder.docx</code>	Email / review comments
PowerPoint	<code>docs/hld-stakeholder.pptx</code>	Live stakeholder presentation
LLD	<code>docs/lld.html</code> / <code>docs/lld.pdf</code>	Engineering detail

Regenerate: `npm run docs:pdf` and `npm run docs:hld:stakeholder`.

15. Related documents

See [Low Level Design \(LLD\)](#) for module contracts, hook behavior, artifact flags, action API groups, self-heal, error catalog, database schema, LLM routing, and extension procedures. Interactive summaries are also available as Cursor canvases ([h1d-architecture](#), [11d-modules](#)).